

Guía Docente y Solucionario Explicado: Actividades de Aula

UT1: Manejo de Ficheros (Texto, Binarios y XML) · Módulo Acceso a Datos (DAM2)

MÓDULO CURRICULAR Acceso a Datos (0486)	UNIDAD DIDÁCTICA UT1 / UD01 (Persistencia)	DOCENTE RESPONSABLE Sergio Capdevila Díez
ESTILO DE CÓDIGO Java accesible y didáctico		

Bloque 1: Ficheros de Texto Plano (java.io y Java NIO.2)

Actividad 1.1: Diagnóstico de Charset y Conversión ISO-8859-1 a UTF-8

SLIDE 03 · APUNTES 1.0

Objetivo en el aula: Demostrar que los bytes en disco requieren una tabla de decodificación explícita. Evitar el fenómeno del *Mojibake* (caracteres corruptos como Ã±).

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.*;
import java.nio.charset.StandardCharsets;

public class SolucionActividad1_1 {
    public static void main(String[] args) {
        File origen = new File("datos/usuarios_legacy.txt");
        File destino = new File("datos/usuarios_utf8.txt");

        // 1. Abrimos leyendo en ISO-8859-1 y volcando en UTF-8
        try (
            BufferedReader lector = new BufferedReader(
                new InputStreamReader(new FileInputStream(origen), StandardCharsets.ISO_8859_1)
            );
            BufferedWriter escritor = new BufferedWriter(
                new OutputStreamWriter(new FileOutputStream(destino), StandardCharsets.UTF_8)
            )
        ) {
            String linea;
            int contador = 0;
            while ((linea = lector.readLine()) != null) {
                escritor.write(linea);
                escritor.newLine(); // Salto de línea adecuado al SO
                contador++;
            }
            System.out.println("✅ Conversión finalizada con éxito. Líneas procesadas: " + contador);
        } catch (IOException e) {
            System.err.println("❌ Error I/O: " + e.getMessage());
        }
    }
}
```

EN LA PIZARRA:

Explica el patrón decorador ("el sándwich"):
FileInputStream lee bytes →
InputStreamReader traduce con la tabla elegida
→ BufferedReader aporta el búfer en RAM y
readLine().

ERROR HABITUAL DEL ALUMNO:

Usar `new FileReader(origen)` sin especificar charset. En Windows funciona por azar si coincide con Windows-1252, pero fallará al subir el código a un servidor Linux.

Actividad 1.2: Lectura y Escritura Clásica Optimizada con Búferes

SLIDE 04 · APUNTES 1.1

Objetivo en el aula: Entender por qué el búfer en memoria RAM (8 KB) multiplica por 100 la velocidad de lectura respecto a accesos directos carácter a carácter.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.*;
import java.nio.charset.StandardCharsets;

public class SolucionActividad1_2 {
    public static void main(String[] args) {
        File fIn = new File("datos/servidores.txt");
        File fOut = new File("datos/servidores_backup.txt");

        try (
            BufferedReader br = new BufferedReader(new FileReader(fIn, StandardCharsets.UTF_8));
            BufferedWriter bw = new BufferedWriter(new FileWriter(fOut, StandardCharsets.UTF_8))
        ) {
            String linea;
            int numLinea = 1;
            while ((linea = br.readLine()) != null) {
                bw.write "[" + numLinea + " ] " + linea;
                bw.newLine();
                numLinea++;
            }
            bw.flush(); // Forzamos el vaciado del búfer antes de salir
            System.out.println("✅ Copia y numeración completada. Total: " + (numLinea - 1));
        } catch (IOException e) {
            System.err.println("❌ Error: " + e.getMessage());
        }
    }
}
```

EN LA PIZARRA:

Haz la analogía del albañil: llevar ladrillos uno a uno con la mano (sin búfer) vs usar una carretilla (con búfer). El disco mecánico o SSD agradece lecturas en bloques.

ERROR HABITUAL DEL ALUMNO:

Olvidar `bw.newLine()`. Si no se llama, todo el fichero de destino se concatena en una sola línea continua.

Actividad 1.3: Generador de Informes Formateados con PrintWriter

SLIDE 05 · APUNTES 1.2

Objetivo en el aula: Aplicar `printf()` con especificadores de formato para generar tablas tabuladas limpias directamente a fichero de texto.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.*;
import java.nio.charset.StandardCharsets;

public class SolucionActividad1_3 {
    public static void main(String[] args) {
        File informe = new File("datos/informe_ventas.txt");
        String[] productos = { "Teclado RGB", "Ratón Óptico", "Monitor 4K 27'" };
        int[] unidades = { 15, 32, 8 };
        double[] precios = { 79.99, 45.50, 289.00 };

        try (PrintWriter pw = new PrintWriter(new FileWriter(informe, StandardCharsets.UTF_8))) {
            pw.println("=====");
            pw.println("                INFORME SEMANAL DE VENTAS                ");
            pw.println("=====");
            pw.printf("%-25s | %8s | %10s | %10s%n", "PRODUCTO", "UDS", "PRECIO", "TOTAL");
            pw.println("-----");

            double totalFactura = 0;
            for (int i = 0; i < productos.length; i++) {
                double subtotal = unidades[i] * precios[i];
                totalFactura += subtotal;
                pw.printf("%-25s | %8d | %9.2f€ | %9.2f€%n",
                    productos[i], unidades[i], precios[i], subtotal);
            }
            pw.println("=====");
            pw.printf("%-25s  %8s  %10s | %9.2f€%n", "TOTAL GLOBAL", "", "", totalFactura);
            System.out.println("✅ Informe generado en: " + informe.getAbsolutePath());
        } catch (IOException e) {
            System.err.println("❌ Fallo al generar informe: " + e.getMessage());
        }
    }
}
```

EN LA PIZARRA:

Explica que `%-25s` alinea a la izquierda ocupando 25 espacios y `%9.2f` reserva 9 huecos con 2 decimales a la derecha. Usa `%n` para salto de línea universal.

ERROR HABITUAL DEL ALUMNO:

Crear que `PrintWriter` avisa con excepciones. Como silencia las `IOException`, hay que comprobar `pw.checkError()` si se sospecha fallo de disco.

Actividad 1.4: Operaciones Rápidas con Java NIO.2 (Files)

SLIDE 06 · APUNTES 1.3

Objetivo en el aula: Comprobar la potencia de la clase utilitaria Files de Java NIO.2 para leer y escribir ficheros pequeños/medianos en una sola línea.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.List;

public class SolucionActividad1_4 {
    public static void main(String[] args) {
        Path ruta = Paths.get("datos/app_config.json");

        try {
            // 1. Escritura directa de un String completo en 1 línea
            String json = "{\n  \"puerto\": 8080,\n  \"ambiente\": \"desarrollo\"\n}";
            Files.writeString(ruta, json, StandardCharsets.UTF_8);
            System.out.println("✅ Archivo JSON creado en una sola llamada.");

            // 2. Lectura directa completa como String
            String contenido = Files.readString(ruta, StandardCharsets.UTF_8);
            System.out.println("📄 Contenido leído:\n" + contenido);

            // 3. Añadir línea al final usando StandardOpenOption.APPEND
            Files.writeString(ruta, "\n// Fin de fichero", StandardCharsets.UTF_8, StandardOpenOption.APPEND);
            System.out.println("✅ Línea añadida con StandardOpenOption.APPEND.");
        } catch (IOException e) {
            System.err.println("❌ Error en Java NIO.2: " + e.getMessage());
        }
    }
}
```

EN LA PIZARRA:

Resalta la concisión: adiós a las 10 líneas de `try`, `new FileWriter`, `new BufferedWriter` y `close()`. Ideal para ficheros de configuración `.json`, `.properties` o `.xml`.

ADVERTENCIA PARA EL ALUMNADO:

Jamás usar `Files.readString()` con archivos de 2 GB. Provocaría un `OutOfMemoryError` inmediato al intentar volcar 2 GB continuos en la pila de memoria.

Actividad 1.5: Filtro de Logs en Streaming con Files.lines()

SLIDE 07 · APUNTES 1.4

Objetivo en el aula: Procesar ficheros de gran tamaño sin saturar la RAM mediante evaluación perezosa (*lazy*) de Java Streams.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.stream.Stream;

public class SolucionActividad1_5 {
    public static void main(String[] args) {
        Path origen = Paths.get("datos/servidor.log");
        Path destino = Paths.get("datos/solo_errores.log");

        // IMPORTANTE: El Stream abre un descriptor de SO; DEBE ir dentro del try()
        try (
            Stream lineas = Files.lines(origen, StandardCharsets.UTF_8);
            BufferedWriter bw = Files.newBufferedWriter(destino, StandardCharsets.UTF_8)
        ) {
            lineas
                .filter(l -> l.contains("[ERROR]") || l.contains("[FATAL]"))
                .forEach(l -> {
                    try {
                        bw.write(l);
                        bw.newLine();
                    } catch (IOException ex) {
                        System.err.println("Error escribiendo: " + ex.getMessage());
                    }
                });
            System.out.println("✅ Filtrado finalizado. Errores guardados en: " + destino);
        } catch (IOException e) {
            System.err.println("❌ Error: " + e.getMessage());
        }
    }
}
```

EN LA PIZARRA:

Explica la diferencia entre un lago (memoria con todos los datos) y una tubería con grifo (streaming). `Files.lines()` solo tiene 1 línea en memoria a la vez.

ERROR HABITUAL DEL ALUMNO:

Olvidar que Stream implementa AutoCloseable. Si no se declara en el try(...), el fichero queda bloqueado en Windows impidiendo su borrado.

Bloque 2: Ficheros Binarios de Datos Simples y Objetos

Actividad 2.1: Flujos Binarios Primitivos y Control Canónico de EOFException

SLIDE 09 · APUNTES 2.1

Objetivo en el aula: Comprender que `DataInputStream` no devuelve `null` ni `-1`, sino que avisa del fin de archivo lanzando `EOFException`.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.*;

public class SolucionActividad2_1 {
    public static void main(String[] args) {
        File binario = new File("datos/telemetry.dat");

        // 1. ESCRITURA DE DATOS PRIMITIVOS
        try (DataOutputStream dos = new DataOutputStream(new FileOutputStream(binario))) {
            dos.writeInt(101); dos.writeDouble(23.5); dos.writeBoolean(false);
            dos.writeInt(102); dos.writeDouble(52.1); dos.writeBoolean(true); // Alarma
            System.out.println("✅ Escritos 2 registros. Tamaño: " + binario.length() + " bytes");
        } catch (IOException e) {
            System.err.println("Error escritura: " + e.getMessage());
        }

        // 2. LECTURA CON BUCLE INFINITO Y CONTROL DE EOFException
        try (DataInputStream dis = new DataInputStream(new FileInputStream(binario))) {
            while (true) {
                int id = dis.readInt();
                double temp = dis.readDouble();
                boolean alarma = dis.readBoolean();
                System.out.printf("-> Sensor %d: %.2f °C | Alarma: %s\n", id, temp, (alarma ? "Sí" : "NO"));
            }
        } catch (EOFException e) {
            // ¡ESTO NO ES UN ERROR! Es el fin natural del archivo binario
            System.out.println("❌ Fin de fichero alcanzado correctamente (EOFException capturada).");
        } catch (IOException e) {
            System.err.println("Error lectura inesperado: " + e.getMessage());
        }
    }
}
```

EN LA PIZARRA:

Haz que calculen el tamaño: 2 registros $\times$ (4 bytes del `int` + 8 del `double` + 1 del `boolean`) = 26 bytes exactos.
Cero sobrecoste de parseo.

ERROR HABITUAL DEL ALUMNO:

Alterar el orden de lectura (ej. leer `readDouble()` antes de `readInt()`). Explicar que el puntero leerá bytes mezclados produciendo números sin sentido.

Actividad 2.2: Almacén de Registros de Longitud Fija con RandomAccessFile

SLIDE 11 · APUNTES 2.2

Objetivo en el aula: Aprender a diseñar registros de longitud constante en disco para poder calcular offsets matemáticos con seek().

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.*;

public class SolucionActividad2_2 {
    // int id (4) + 10 chars (20) + double precio (8) = 32 bytes
    public static final int TAMANO_REG = 32;

    public static void main(String[] args) {
        File f = new File("datos/almacen_raf.dat");
        try (RandomAccessFile raf = new RandomAccessFile(f, "rw")) {
            escribirReg(raf, 0, 1, "Teclado", 49.99);
            escribirReg(raf, 1, 2, "Raton", 19.50);
            escribirReg(raf, 2, 3, "Auricular", 35.00);
            System.out.println("✅ 3 registros guardados. Total: " + raf.length() + " bytes");
        } catch (IOException e) {
            System.err.println("Error RAF: " + e.getMessage());
        }
    }

    public static void escribirReg(RandomAccessFile raf, int pos, int id, String nom, double pre) throws
        raf.seek((long) pos * TAMANO_REG);
        raf.writeInt(id);
        StringBuffer sb = new StringBuffer(nom);
        sb.setLength(10); // Fuerza exactamente 10 caracteres
        raf.writeChars(sb.toString());
        raf.writeDouble(pre);
    }
}
```

EN LA PIZARRA:

Dibuja los casilleros de 32 bytes: Posición 0 (offset 0), Posición 1 (offset 32), Posición 2 (offset 64). La fórmula:
$$\text{Offset} = \text{Posición} \times 32$$

ERROR HABITUAL DEL ALUMNO:

Intentar usar `raf.writeUTF(nom)`. `writeUTF` tiene tamaño dinámico y descuadra los offsets de los registros siguientes.

Actividad 2.3: Actualización In-Situ de Stock con RandomAccessFile

SLIDE 12 · APUNTES 2.2

Objetivo en el aula: Comprobar cómo sobrescribir bytes en el medio del archivo sin leer ni alterar los registros anteriores o posteriores.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.*;

public class SolucionActividad2_3 {
    public static void main(String[] args) {
        File f = new File("datos/almacen_raf.dat");
        int posAEditar = 1; // Segundo producto ("Raton")

        try (RandomAccessFile raf = new RandomAccessFile(f, "rw")) {
            // Offset precio = (pos * 32) + 4 (id) + 20 (nombre) = (pos * 32) + 24
            long offsetPrecio = ((long) posAEditar * SolucionActividad2_2.TAMANO_REG) + 24;

            raf.seek(offsetPrecio);
            double anterior = raf.readDouble();
            System.out.printf("Precio anterior: %.2f €\n", anterior);

            // Retrocedemos el puntero para sobrescribir los 8 bytes del double
            raf.seek(offsetPrecio);
            raf.writeDouble(24.95);
            System.out.println("✅ Precio actualizado in-situ a 24.95 € sin tocar el resto del fichero.")
        } catch (IOException e) {
            System.err.println("Error in-situ: " + e.getMessage());
        }
    }
}
```

EN LA PIZARRA:

Explica que la lectura `readDouble()` avanza el puntero 8 bytes hacia adelante. Antes de escribir el nuevo valor, es imperativo volver a llamar a `seek()`.

ERROR HABITUAL DEL ALUMNO:

Olvidar el segundo `seek()`. Si no se reposiciona, el nuevo dato se escribe pisando el siguiente registro contiguo en el archivo.

Actividad 2.4: Serialización de Objetos con Campos Transient

SLIDE 13 · APUNTES 2.3

Objetivo en el aula: Serializar instancias completas con `ObjectOutputStream` y constatar cómo `transient` protege contraseñas y tokens.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.*;

class Usuario implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String login;
    private transient String token; // NO se serializa

    public Usuario(int id, String login, String token) {
        this.id = id; this.login = login; this.token = token;
    }
    public String toString() {
        return "Usuario[" + id + ", login=" + login + ", token=" + (token == null ? "NULL (Protegido)" :
    }
}

public class SolucionActividad2_4 {
    public static void main(String[] args) {
        File f = new File("datos/usuarios.bin");
        Usuario u = new Usuario(1, "sergio", "JWT_SECRET_TOKEN_XYZ");

        // 1. SERIALIZACIÓN
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(f))) {
            oos.writeObject(u);
            System.out.println("✅ Usuario serializado en disco.");
        } catch (IOException e) { e.printStackTrace(); }

        // 2. DESERIALIZACIÓN
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(f))) {
            Usuario uLeido = (Usuario) ois.readObject();
            System.out.println("📄 Usuario recuperado: " + uLeido);
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

EN LA PIZARRA:

Explica el rol de `serialVersionUID`. Si añades un campo a la clase y no tienes fijado el UID, Java lanzará `InvalidClassException` al deserializar.

ERROR HABITUAL DEL ALUMNO:

Olvidar el casting explícito `(Usuario) ois.readObject()`, provocando un error de compilación de tipos incompatibles.

Actividad 2.5: El Gran Reto - Persistencia Incremental sin Corrupción (MiObjectOutputStream)

SLIDES 14 Y 15 · APUNTES 2.4

Objetivo en el aula: Resolver el clásico problema del modo append en serialización, anulando la cabecera repetida 0xACED0005.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.*;

// Subclase que no escribe cabecera intermedia
class MiObjectOutputStream extends ObjectOutputStream {
    public MiObjectOutputStream(OutputStream out) throws IOException { super(out); }
    @Override
    protected void writeStreamHeader() throws IOException {
        // Método vacío intencionado: preserva la cabecera original del fichero
    }
}

public class SolucionActividad2_5 {
    private static final File F = new File("datos/usuarios_append.bin");

    public static void main(String[] args) {
        if (F.exists()) F.delete();
        // Insertamos 2 usuarios en llamadas separadas
        guardarUsuario(new Usuario(10, "carlos", "token1"));
        guardarUsuario(new Usuario(20, "maria", "token2"));
        leerTodos();
    }

    public static void guardarUsuario(Usuario u) {
        boolean yaExiste = F.exists() && F.length() > 0;
        try {
            FileOutputStream fos = new FileOutputStream(F, true);
            ObjectOutputStream oos = yaExiste ? new MiObjectOutputStream(fos) : new ObjectOutputStream(fos);
        } {
            oos.writeObject(u);
            System.out.println("-> Guardado: " + u + " (¿Usó MiOOS?: " + yaExiste + ")");
        } catch (IOException e) { e.printStackTrace(); }
    }

    public static void leerTodos() {
        System.out.println("\n📖 Leyendo registros:");
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(F))) {
            while (true) {
                Usuario u = (Usuario) ois.readObject();
                System.out.println("  " + u);
            }
        } catch (EOFException e) {
            System.out.println("🏁 Fin alcanzado sin StreamCorruptedException.");
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

DEMOSTRACIÓN DE IMPACTO:

Haz que usen `new ObjectOutputStream` en ambas inserciones. Verán el error: `StreamCorruptedException`:

PUNTOS A REVISAR EN LAS MESAS:

Verificar la condición ternaria: si el fichero mide 0 bytes, hay que usar el `ObjectOutputStream` original para que escriba

invalid type code: AC. Explica que `AC` son los primeros bytes de la cabecera repetida.

la cabecera inicial imprescindible.

Bloque 3: Ficheros XML (DOM, SAX y JAXB)

Actividad 3.1: Consulta y Modificación de Catálogo con DOM

SLIDE 17 · APUNTES 3.1

Objetivo en el aula: Cargar un XML en memoria RAM, filtrar nodos fantasma de tipo #text (espacios y saltos de línea) y modificar valores in-situ.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.File;
import javax.xml.parsers.*;
import org.w3c.dom.*;

public class SolucionActividad3_1 {
    public static void main(String[] args) throws Exception {
        File f = new File("datos/catalogo.xml");
        DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
        DocumentBuilder builder = factory.newDocumentBuilder();
        Document doc = builder.parse(f);

        doc.getDocumentElement().normalize(); // Limpieza obligatoria
        NodeList lista = doc.getElementsByTagName("producto");

        for (int i = 0; i < lista.getLength(); i++) {
            Node n = lista.item(i);
            // FILTRADO Estricto: Ignorar nodos de texto intermedios (#text)
            if (n.getNodeType() == Node.ELEMENT_NODE) {
                Element e = (Element) n;
                String id = e.getAttribute("id");
                Element elemPrecio = (Element) e.getElementsByTagName("precio").item(0);
                double precio = Double.parseDouble(elemPrecio.getTextContent());
                double nuevoPrecio = precio * 1.10; // Subida del 10%
                elemPrecio.setTextContent(String.format("%.2f", nuevoPrecio).replace(',', '.'));
                System.out.printf("Producto %s: %.2f -> %.2f €\n", id, precio, nuevoPrecio);
            }
        }
        System.out.println("✅ Árbol DOM modificado en memoria RAM.");
    }
}
```

EN LA PIZARRA:

Dibuja el árbol DOM completo. Destaca que entre `<catalogo>` y `<producto>` el salto de línea es interpretado como un nodo hijo de tipo `#text`.

ERROR HABITUAL DEL ALUMNO:

Hacer un cast a `(Element)` sin comprobar `n.getNodeType() == Node.ELEMENT_NODE`, provocando un `ClassCastException` inmediato.

Actividad 3.2: Generación de XML y Volcado Formateado con Transformer

SLIDE 18 · APUNTES 3.1

Objetivo en el aula: Construir un árbol DOM en blanco mediante `doc.createElement()` y serializarlo a disco con sangría legible (*pretty-print*).

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.File;
import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import org.w3c.dom.*;

public class SolucionActividad3_2 {
    public static void main(String[] args) throws Exception {
        DocumentBuilder builder = DocumentBuilderFactory.newInstance().newDocumentBuilder();
        Document doc = builder.newDocument();

        // Creamos la raíz y los hijos
        Element raiz = doc.createElement("alumnos");
        doc.appendChild(raiz);

        Element alu = doc.createElement("alumno");
        alu.setAttribute("expediente", "EXP-001");

        Element nom = doc.createElement("nombre");
        nom.setTextContent("Lucía Fernández");
        alu.appendChild(nom);
        raiz.appendChild(alu);

        // Volcado a disco formateado con Transformer
        Transformer tf = TransformerFactory.newInstance().newTransformer();
        tf.setOutputProperty(OutputKeys.INDENT, "yes");
        tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");

        tf.transform(new DOMSource(doc), new StreamResult(new File("datos/alumnos.xml")));
        System.out.println("✅ Archivo XML generado con sangría de 4 espacios.");
    }
}
```

EN LA PIZARRA:

Explica la regla de oro: los elementos XML no se instancian con `new Element()`. Pertenecen obligatoriamente a la factoría del documento con `doc.createElement()`.

ERROR HABITUAL DEL ALUMNO:

Omitir las propiedades de sangría. Si no se activan, todo el XML se escribe en una única línea infinita sin espacios en blanco.

Actividad 3.3: Analizador Reactivo sin Memoria con SAX

SLIDE 19 · APUNTES 3.2

Objetivo en el aula: Implementar un DefaultHandler para procesar gigabytes de XML a velocidad de vértigo y con consumo de RAM prácticamente nulo.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import java.io.File;
import javax.xml.parsers.*;
import org.xml.sax.*;
import org.xml.sax.helpers.DefaultHandler;

public class SolucionActividad3_3 {
    public static void main(String[] args) throws Exception {
        SAXParser parser = SAXParserFactory.newInstance().newSAXParser();
        HandlerStats handler = new HandlerStats();

        parser.parse(new File("datos/catalogo.xml"), handler);

        System.out.println("📄 Resumen SAX:");
        System.out.println("  Total productos: " + handler.total);
        System.out.printf("  Media precios: %.2f €%n", (handler.suma / handler.total));
    }
}

class HandlerStats extends DefaultHandler {
    StringBuilder sb = new StringBuilder();
    int total = 0; double suma = 0; boolean enPrecio = false;

    public void startElement(String u, String l, String q, Attributes atts) {
        sb.setLength(0); // Limpiamos acumulador
        if (q.equalsIgnoreCase("producto")) total++;
        if (q.equalsIgnoreCase("precio")) enPrecio = true;
    }
    public void characters(char[] ch, int s, int len) {
        sb.append(ch, s, len); // ¡Acumulamos siempre!
    }
    public void endElement(String u, String l, String q) {
        if (q.equalsIgnoreCase("precio") && enPrecio) {
            suma += Double.parseDouble(sb.toString().trim());
            enPrecio = false;
        }
    }
}
```

LA TRAMPA DE CHARACTERS():

Explica con énfasis por qué no se puede hacer un `parseDouble()` directamente en `characters()`. El parser puede trocear el número y llamar a `characters()` 2 o 3 veces seguidas.

ESCENARIO LABORAL REAL:

DOM es ideal para modificar documentos pequeños (< 20 MB). SAX es el estándar para facturas masivas del banco o aduanas de 5 GB.

Actividad 3.4: Mapeo Declarativo Objeto-XML con JAXB

SLIDE 20 · APUNTES 3.4

Objetivo en el aula: Automatizar la conversión entre objetos POJO y XML mediante anotaciones `@XmlRootElement` y `@XmlElement` sin tocar nodos.

CÓDIGO JAVA RESUELTO:

```
package es.cifpaviles.ad.ut1.soluciones;

import jakarta.xml.bind.JAXBContext;
import jakarta.xml.bind.Marshaller;
import jakarta.xml.bind.Unmarshaller;
import jakarta.xml.bind.annotation.*;
import java.io.File;

@XmlRootElement(name = "curso")
class Curso {
    private String codigo;
    private String nombre;

    public Curso() {} // ¡OBLIGATORIO para JAXB!
    public Curso(String c, String n) { this.codigo = c; this.nombre = n; }

    @XmlAttribute public String getCodigo() { return codigo; }
    public void setCodigo(String c) { this.codigo = c; }

    @XmlElement public String getNombre() { return nombre; }
    public void setNombre(String n) { this.nombre = n; }

    public String toString() { return "Curso[" + codigo + " - " + nombre + "];" }
}

public class SolucionActividad3_4 {
    public static void main(String[] args) throws Exception {
        File f = new File("datos/curso.xml");
        JAXBContext ctx = JAXBContext.newInstance(Curso.class);

        // 1. MARSHALLER: Objeto -> XML
        Marshaller m = ctx.createMarshaller();
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
        m.marshal(new Curso("DAM2", "Acceso a Datos"), f);
        System.out.println("✅ XML generado con Marshaller.");

        // 2. UNMARSHALLER: XML -> Objeto
        Unmarshaller u = ctx.createUnmarshaller();
        Curso leido = (Curso) u.unmarshal(f);
        System.out.println("📄 Objeto recuperado: " + leido);
    }
}
```

EN LA PIZARRA:

Destaca que `Marshaller` hace la salida (Java a XML) y `Unmarshaller` hace la entrada (XML a Java). Asocia los nombres con el empaquetado de datos.

ERROR CRÍTICO DEL ALUMNO:

Olvidar el constructor por defecto `public Curso() {}`. Si se omite, JAXB no puede instanciar el objeto y lanzará un fallo en ejecución.

Matriz de Orientación Docente

Bloque	Actividades	Concepto Didáctico Clave	Principal Dificultad del Alumno
1. Texto	1.1 a 1.5	UTF-8 innegociable, búferes en RAM y streaming con <code>Files.lines()</code> .	Distinguir cuándo usar <code>Files.readString()</code> vs <code>Files.lines()</code> .
2. Binarios	2.1 a 2.5	Offsets fijos con RAF y resolución del append con <code>MiOutputStream</code> .	La cabecera repetida <code>0xACED0005</code> y el retroceso del puntero con <code>seek()</code> .
3. XML	3.1 a 3.4	Árbol DOM vs eventos SAX vs POJO declarativo con JAXB.	El filtrado de <code>#text</code> en DOM y el acumulador <code>StringBuilder</code> en SAX.